

Week 7 Lab: RTOS Communication - Sensor Pipeline

Overview

Build a multi-stage sensor data pipeline using FreeRTOS queues and mutexes. This lab demonstrates proper inter-task communication by reading ADC values, processing them through a queue-based pipeline, and outputting results via UART with mutex protection.

Time allocation: 90 minutes

- CubeMX project setup: 20 min
- Queue and mutex implementation: 30 min
- Pipeline testing and debugging: 25 min
- Failure injection experiments: 15 min

Learning Objectives

By the end of this lab, you will be able to:

1. Create and configure message queues in FreeRTOS
2. Implement producer-consumer pattern with queues
3. Protect shared resources (UART) using mutexes
4. Build multi-stage data processing pipelines
5. Debug queue overflow and mutex contention issues
6. Observe the difference between protected and unprotected output

Safety Gate

Safety Checklist

This lab uses low-voltage components only.

- ☐ USB cable connected to Nucleo board
- ☐ Potentiometer wired correctly (no shorts)
- ☐ Bench supply OFF (USB power is sufficient)

Good Habits

- Verify potentiometer connections before power-on
- Check ADC pin is configured as analog input
- Confirm UART settings match terminal (115200 baud)

Required Parts and Tools

Hardware

- STM32 Nucleo-F401RE board
- Potentiometer (10k Ω) from kit
- Breadboard and jumper wires
- USB cable (for power and UART)

Software

- STM32CubeIDE with FreeRTOS
- Serial terminal (PuTTY, Tera Term, or CubeIDE terminal)

! PRE-LAB SECTION

Complete the following **BEFORE** arriving at the lab session.

Pre-Lab Preparation

1. Queue Sizing Calculation

Given: - SensorTask produces 10 readings/second - ProcessTask consumes 1 reading every 100ms (10/second average) - But ProcessTask batches 4 readings before output

Question 1: What is the minimum queue size to prevent overflow?

Calculation: _____

Recommended size: _____ items

2. Mutex Understanding

Question 2: What happens if two tasks call printf() simultaneously without mutex protection?

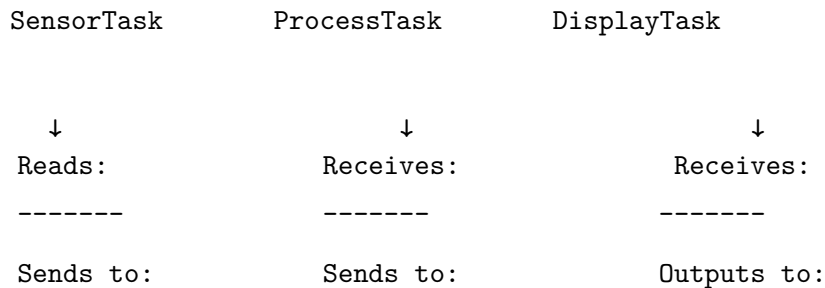
Answer: _____

Question 3: Why can't we use a mutex inside an ISR?

Answer: _____

3. Data Flow Diagram

Draw the data flow for this pipeline:



i IN-LAB SECTION

The following sections are completed during the lab session with TA supervision.

Part 1: Hardware Setup (10 minutes)

Step 1.1: Connect Potentiometer

Wire the potentiometer as a voltage divider:

Potentiometer Pin	Connect To
Pin 1 (end)	3.3V
Pin 2 (wiper/middle)	PA0
Pin 3 (end)	GND

Verify with multimeter: Rotate pot, voltage at PA0 should vary 0V to 3.3V

Step 1.2: UART Connection

The Nucleo board has built-in ST-Link virtual COM port:

- **TX:** PA2 (USART2_TX)
- **RX:** PA3 (USART2_RX)
- **Baud:** 115200

Open terminal: Connect to the COM port that appears when Nucleo is plugged in.

Part 2: CubeMX Configuration (20 minutes)

Step 2.1: Create New Project

1. File → New → STM32 Project
2. Select board: NUCLEO-F401RE
3. Project name: W07_Pipeline_Lab

Step 2.2: Configure ADC

1. Analog → ADC1 → IN0
2. Parameter Settings:
 - Resolution: 12 bits
 - Scan Conversion Mode: Disabled
 - Continuous Conversion Mode: Disabled
 - Discontinuous Conversion Mode: Disabled
 - DMA Continuous Requests: Disabled

Verify: PA0 should show as ADC1_IN0 in pinout view

Step 2.3: Configure USART2

1. Connectivity → USART2 → Mode: Asynchronous
2. Parameter Settings:
 - Baud Rate: 115200
 - Word Length: 8 Bits
 - Stop Bits: 1
 - Parity: None

Step 2.4: Enable FreeRTOS

1. Middleware → FREERTOS → CMSIS_V2
2. Config parameters:
 - TOTAL_HEAP_SIZE: 15360
 - USE_NEWLIB_REENTRANT: Enabled (for printf)

Step 2.5: Create Queues

In Queues tab, add:

Queue Name	Queue Size	Item Size
RawDataQueue	10	2 (uint16_t)
ProcessedQueue	5	2 (uint16_t)

Step 2.6: Create Mutex

In **Mutexes** tab, add:

Mutex Name	Type
UartMutex	Normal

Step 2.7: Create Tasks

Task Name	Priority	Stack	Entry Function
SensorTask	osPriorityNormal	128	StartSensorTask
ProcessTask	osPriorityNormal	128	StartProcessTask
DisplayTask	osPriorityBelowNormal	256	StartDisplayTask

Note: DisplayTask needs larger stack for printf

Step 2.8: Generate Code

Save and generate code. Open project in CubeIDE.

Part 3: Implement Tasks (30 minutes)

Step 3.1: Add Required Includes

At the top of `freertos.c` (in USER CODE section):

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

Step 3.2: Redirect printf to UART

Add to `freertos.c` (or create separate file):

```
/* USER CODE BEGIN 0 */
// Redirect printf to UART2
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}
/* USER CODE END 0 */
```

Note: Also add `extern UART_HandleTypeDef huart2;` if needed.

Step 3.3: Implement SensorTask

```
void StartSensorTask(void *argument)
{
    /* Sensor task: Read ADC and send to queue
     * Samples at 10 Hz (100ms interval)
     */
    uint16_t adc_value;
    uint32_t sample_count = 0;

    for(;;)
    {
        // Start ADC conversion
        HAL_ADC_Start(&hadc1);
```

```

    // Wait for conversion (with timeout)
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
    {
        // Read ADC value
        adc_value = HAL_ADC_GetValue(&hadc1);

        // Send to queue (non-blocking with timeout)
        osStatus_t status = osMessageQueuePut(
            RawDataQueueHandle,
            &adc_value,
            0,          // Priority (not used for queues)
            10          // 10ms timeout
        );

        if (status != osOK) {
            // Queue full - data lost!
            // Could increment error counter here
        }

        sample_count++;
    }

    HAL_ADC_Stop(&hadc1);

    osDelay(100); // 10 Hz sample rate
}
}

```

Step 3.4: Implement ProcessTask

```

void StartProcessTask(void *argument)
{
    /* Process task: Receive raw data, compute 4-sample average
     * Outputs one averaged value for every 4 inputs
     */
    uint16_t raw_value;
    uint32_t sum = 0;
    uint8_t count = 0;
    uint16_t average;

```

```

for(;;)
{
    // Block waiting for data from sensor task
    osStatus_t status = osMessageQueueGet(
        RawDataQueueHandle,
        &raw_value,
        NULL,          // Message priority (output)
        osWaitForever  // Block indefinitely
    );

    if (status == osOK)
    {
        // Accumulate for averaging
        sum += raw_value;
        count++;

        // Output average every 4 samples
        if (count >= 4)
        {
            average = sum / 4;

            // Send averaged value to display queue
            osMessageQueuePut(
                ProcessedQueueHandle,
                &average,
                0,
                10 // 10ms timeout
            );

            // Reset accumulator
            sum = 0;
            count = 0;
        }
    }
}
}

```

Step 3.5: Implement DisplayTask

```

void StartDisplayTask(void *argument)
{
    /* Display task: Receive processed data, output via UART
     * Protected by mutex to prevent garbled output
     */
    uint16_t avg_value;
    uint32_t display_count = 0;
    float voltage;

    for(;;)
    {
        // Block waiting for processed data
        osStatus_t status = osMessageQueueGet(
            ProcessedQueueHandle,
            &avg_value,
            NULL,
            osWaitForever
        );

        if (status == osOK)
        {
            display_count++;

            // Convert to voltage (3.3V reference, 12-bit ADC)
            voltage = (avg_value / 4095.0f) * 3.3f;

            // PROTECT UART OUTPUT WITH MUTEX
            osMutexAcquire(UartMutexHandle, osWaitForever);

            printf("[%04lu] ADC Avg: %4u  Voltage: %.2fV\r\n",
                display_count, avg_value, voltage);

            osMutexRelease(UartMutexHandle);
        }
    }
}

```

Step 3.6: Build and Test

1. Build project (Ctrl+B)
2. Flash to board (F11 for debug, or Run)

3. Open serial terminal at 115200 baud
4. Rotate potentiometer - observe values changing

Expected output:

```
[0001] ADC Avg: 2048 Voltage: 1.65V
[0002] ADC Avg: 2050 Voltage: 1.65V
[0003] ADC Avg: 3000 Voltage: 2.42V
...
```

Checkpoint: TA verifies pipeline is working:

Part 4: Debugging and Observation (15 minutes)

Step 4.1: Monitor Queue Status

Add debug output to SensorTask (inside mutex protection):

```
// Add to SensorTask, periodically:
static uint32_t debug_counter = 0;
if (++debug_counter >= 50) { // Every 5 seconds
    uint32_t queue_count = osMessageQueueGetCount(RawDataQueueHandle);

    osMutexAcquire(UartMutexHandle, osWaitForever);
    printf("[DEBUG] RawQueue items: %lu\r\n", queue_count);
    osMutexRelease(UartMutexHandle);

    debug_counter = 0;
}
```

Observation: Queue should have 0-4 items typically. If consistently near 10 (max), consumer is too slow.

Step 4.2: Test Without Mutex (Demonstrate Problem)

TEMPORARILY comment out mutex in DisplayTask:

```
// osMutexAcquire(UartMutexHandle, osWaitForever);
printf(...);
// osMutexRelease(UartMutexHandle);
```

Add another task that also prints:

```
// Add to SensorTask main loop:
// osMutexAcquire(UartMutexHandle, osWaitForever);
printf("S"); // Just one character without mutex
// osMutexRelease(UartMutexHandle);
```

Observe output: Should see garbled/interleaved text!

Without Mutex	With Mutex
[00S01] ADCS Avg: 20S48 Garbled	[0001] ADC Avg: 2048 Clean

Restore mutex protection before continuing!

Checkpoint: TA verifies garbled output demonstrated:

Step 4.3: Queue Overflow Test

TEMPORARILY reduce queue size:

In CubeMX, change RawDataQueue size from 10 to 2. Regenerate code and test.

Add overflow detection:

```
osStatus_t status = osMessageQueuePut(...);
if (status == osErrorTimeout) {
    // Queue overflow!
    overflow_count++;
}
```

Observe: With queue size 2, overflows should occur.

Restore queue size to 10 before continuing!

Part 5: Failure Injection Tests (15 minutes)

Test 1: Remove osDelay from SensorTask

Predict: What happens if SensorTask runs continuously without delay?

Prediction: _____

Test: Comment out `osDelay(100);` in SensorTask. Observe behavior.

Result: _____

Restore the delay!

Test 2: Block ProcessTask

Predict: What happens if ProcessTask stops consuming from queue?

Prediction: _____

Test: Add `osDelay(10000);` at start of ProcessTask loop. Observe.

Result: _____

Remove the blocking delay!

Test 3: Deadlock Scenario

Would this code deadlock?

```
// Task A
osMutexAcquire(MutexX, osWaitForever);
osMutexAcquire(MutexY, osWaitForever);
// work
osMutexRelease(MutexY);
osMutexRelease(MutexX);

// Task B
osMutexAcquire(MutexY, osWaitForever);
osMutexAcquire(MutexX, osWaitForever);
// work
osMutexRelease(MutexX);
osMutexRelease(MutexY);
```


Answer: Yes, deadlock possible No, safe

Explanation: _____

Measurement Summary

ID	Measurement	Expected	Measured	Pass?
M1	ADC sample rate	10 Hz		
M2	Averaged output rate	2.5 Hz		
M3	RawQueue typical depth	0-4 items		
M4	ADC noise (variation)	<50 LSB		
M5	Queue overflow (normal)	0		
M6	UART output quality	No garble		

Voltage Readings

Pot Position	Expected ADC	Expected V	Measured ADC	Measured V
Full CCW	~0	~0V		
Mid	~2048	~1.65V		
Full CW	~4095	~3.3V		

Analysis Questions

1. Why does `ProcessTask` use `osWaitForever` timeout while `SensorTask` uses a short timeout?

2. What would happen if we used `HAL_Delay()` instead of `osDelay()` in `SensorTask`?

3. Why is the mutex released immediately after `printf`, not held longer?

4. How would you modify this design to handle 100 Hz sampling instead of 10 Hz?

Deliverables Checklist

- ☐ Working 3-task pipeline (Sensor → Process → Display)
- ☐ UART output shows ADC values and calculated voltage
- ☐ Queue-based data transfer demonstrated
- ☐ Mutex protection demonstrated (showed garbled vs clean)
- ☐ Queue overflow test performed
- ☐ Completed measurement table
- ☐ Answered analysis questions

Grading Rubric

Criterion	Points	Description
Pipeline setup	25	All three tasks running, queues configured
Data flow	25	ADC → Queue → Average → Queue → Display works
Mutex protection	20	UART protected, demonstrated problem without
Debugging	15	Queue monitoring, overflow test completed
Documentation	15	Measurements, analysis questions

Total: 100 points

Common Issues and Solutions

Issue	Symptom	Solution
No UART output	Terminal blank	Check baud rate, <code>_write</code> redirect
Queue overflow	Data lost	Increase queue size or slow producer
Garbled output	Interleaved text	Add mutex protection
ADC reads 0	Always zero	Check ADC clock, GPIO as analog
Tasks don't run	Nothing happens	Check <code>osKernelStart()</code> , handles
Hard fault	Crash	Increase stack size (especially Display)

Cleanup

- ☐ Stop debug session
- ☐ Save project
- ☐ Disconnect potentiometer

- ☐ Close serial terminal
- ☐ Document any issues encountered

Key Takeaways

1. **Queues decouple tasks** - Producer and consumer run independently
2. **Mutexes prevent data corruption** - Essential for shared resources like UART
3. **Always use timeouts** - Prevents deadlocks, enables error detection
4. **Pipeline pattern** - Powerful for multi-stage data processing
5. **Test failure cases** - Understanding failures helps design robust systems